

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

КУРСОВАЯ РАБОТА
по дисциплине «Технологии автоматизации процесса разработки
программного обеспечения»
Вариант 52

Студент гр. 1306

Шаврин А.П.

Преподаватель

Бутенко Д.С.

Санкт-Петербург

2026

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Студент Шаврин А.П.

Группа 1306

Исходные данные:

Требуется реализовать docker-compose конфигурацию из двух контейнеров:

- app — контейнер с веб-приложением из репозитория devops-examples
- tester — контейнер для запуска тестов

Согласно варианту 52 необходимо выполнить: Проверка бьютификатором для JS (pyjshint), анализ по 10 существующим критериям (pylint), интеграционные тесты — проверка на коды возврата, доступ в app по публичному ключу ssh, вывод логов tester stdout+stderr в docker logs и в общие файлы (отдельно для stdout и stderr), список этапов тестирования для запуска задается через .env, Ограничение ядер процессора (1 ядро)

Содержание пояснительной записки:

Постановка задачи, Описание Dockerfile, Описание скриптов запуска тестов, Описание Docker-compose конфигурации, Заключение, Список использованных. Приложение

Предполагаемый объем пояснительной записки:

Не менее 20 страниц.

Студент

Шаврин А.П,

Преподаватель

Бутенко Д.С.

АННОТАЦИЯ

В рамках курсового проекта разработана конфигурация Docker Compose для автоматизации тестирования веб-приложения. Проект включает два контейнера: app с веб-приложением на Flask и tester с набором тестов (статический анализ через pylint, проверка форматирования JavaScript через pyjslint, интеграционные тесты на коды возврата). Реализован SSH-доступ в контейнер app по публичному ключу, настроено логирование тестов с разделением stdout и stderr как в консоль Docker, так и в файлы. Управление набором запускаемых тестов осуществляется через переменную окружения TEST_STEPS. Все компоненты собраны с соблюдением требований к версионированию пакетов и оптимизации слоев Docker.

SUMMARY

As part of this course project, a Docker Compose configuration was developed to automate web application testing. The project includes two containers: an app container with a Flask web application and a tester container with a test suite (static analysis via pylint, JavaScript formatting check via pyjslint, and integration tests for return codes). SSH access to the app container via a public key was implemented, and test logging was configured with separate stdout and stderr data, both in the Docker console and in files. The set of tests to run is controlled via the TEST_STEPS environment variable. All components were built in compliance with package versioning requirements and Docker layer optimization.

СОДЕРЖАНИЕ

Введение	5
Постановка задачи	6-8
1. Описание Dockerfile	9-12
1.1. Описание Dockerfile для контейнера app	9-11
1.2. Описание Dockerfile для контейнера tester	11-12
2. Описание скриптов запуска тестов	13-19
2.1. Общий скрипт запуска тестов	13
2.2. Проверка бьютификакатором для JS	14-16
2.3. Статический анализ по 10 существующим критери	16-17
2.4. Интеграционный тест кодов возврата	18-19
3. Описание конфигурации docker-compose	20-21
Заключение	22
Список использованных источников	23

ВВЕДЕНИЕ

Целью данной работы является разработка docker-compose конфигурации для автоматизации тестирования веб-приложения. В рамках проекта созданы два контейнера: app с веб-приложением на Flask из репозитория devops-examples и tester с набором тестов.

В ходе выполнения работы согласно варианту 52 реализованы следующие задачи. Для контейнера app настроен SSH-доступ по публичному ключу, что позволяет подключаться к работающему приложению извне. Для контейнера tester реализованы три типа тестов: проверка форматирования JavaScript-кода с помощью инструмента pyjshint, статический анализ Python-кода по десяти критериям с использованием pylint, а также интеграционные тесты, проверяющие коды возврата HTTP-запросов.

Логирование каждого теста организовано таким образом, что вывод дублируется в стандартные потоки docker logs и сохраняется в отдельные файлы для stdout и stderr. Управление набором запускаемых тестов осуществляется через переменную окружения TEST_STEPS, задаваемую в файле .env. Для обоих контейнеров установлено ограничение в одно ядро процессора в соответствии с вариантом задания.

Все компоненты собраны с соблюдением требований к фиксации версий устанавливаемых пакетов и оптимизации количества слоев в Dockerfile.

ПОСТАНОВКА ЗАДАЧИ

Необходимо реализовать docker-compose конфигурацию из двух узлов (не больше и не меньше):

- app - контейнер с существующим демонстрационным веб-приложением (https://github.com/moevm/devops-examples/tree/main/EXAMPLE_APP).
 - Устанавливать приложение необходимо скачивая репозиторий <https://github.com/moevm/devops-examples> и копируя файлы из него при сборке вашего контейнера:)
 - Чтобы все заработало, вам придется потратить время и поработать - из коробки может не работать.
 - Возможно, вам для выполнения заданий потребуются фиксы в исходник - делайте для них патчи (<https://man7.org/linux/man-pages/man1/patch.1.html>)
 - Корнем дерева процессов выступает запущенное веб-приложение
- tester - контейнер для запуска всех тестов (состав и особенности тестов задаются в таблице вариантов)
 - Корнем дерева процессов выступает стандартный python http сервер (`python -m http.server 3000`)
 - Этот сервер должен быть запущен в каталоге контейнера, где будет происходить работа тестовых скриптов
 - Тестовые скрипты запускаются через `docker exec`

Оба контейнера должны использовать написанные вами образы, собираемые из локальных Dockerfile. Шаблоны для имен Dockerfile:

- Dockerfile_app
- Dockerfile_tester

Помимо Dockerfile, вам также необходимо сделать файл README.md, содержащий примеры команд для запуска тестов и проверки всей конфигурации

Общие требования

Dockerfile:

- Минимальная версия докера Docker version 19.03.13, build 4484c46d9d

- Базовый образ ubuntu:22.04
- Не использовать Expose
- При установке любых пакетов и программ (в том числе в requirements) ВСЕГДА указывать версии
- Ограничить установку зависимостей apt одной строкой (один RUN)
- Если настройка одной части приложения состоит из нескольких команд → необходимо разместить их в одном слое (в одном RUN)

Docker-compose:

- Минимальная версия docker compose version 1.27.4, build 40524192
- Все должно собираться по команде docker-compose build без sudo
- Не использовать тип сети HOST
- Не отрывать лишних (непредусмотренных заданием) портов
- Не использовать порты хост-машины $\Leftarrow 1024$
- Не открывайте лишний раз порты всему миру (``5000:5000``). Тут, на самом деле, вообще не вижу необходимости их открывать, но если надо, можно открыть на localhost (``«127.0.0.1:5000:5000»``)
- Не используйте автоматически сгенерированные имена контейнеров (вроде ``docker exec 9310_molodtsev_da-test-1...``), они могут измениться. Если вы не указываете ``container_name`` (не надо), можно писать ``docker compose exec test ...``
- Не используйте ``container_name`` в ``docker-compose.yml`` без необходимости. Это только создаст проблемы, если вы где-то попытаетесь поднять такой же docker-compose с таким же названием. Вместо ``docker exec <container-name>`` можно использовать ``docker compose exec <service-name>``, где ``<service-name>`` - ключ в yaml-файле.

Задание согласно варианту 52

В области построения тестов требуется:

- выполнить проверку бьютификатором для JavaScript с использованием инструмента `pyjslint`;

- провести статический анализ кода по десяти существующим критериям с помощью `pylint`, настроив запуск на выбранные критерии и допустив все десять ошибок в коде проекта;
- написать интеграционные тесты с использованием библиотеки `requests`, которые проверяют коды возврата HTTP-запросов.

В части настройки Docker необходимо:

- организовать внешний SSH-доступ в контейнер `app` по существующему публичному ключу, обеспечив доступность порта для SSH снаружи `docker-compose` конфигурации;
- настроить вывод логов работы контейнера `tester` таким образом, чтобы каждый этап тестирования выводился в стандартные потоки вывода и ошибок (`stdout` и `stderr`), видимые через `docker logs`, а также сохранялся в общие файлы, отдельно для `stdout` и отдельно для `stderr`.

В части конфигурации docker-compose необходимо:

- реализовать передачу параметров через файл `.env`, а именно списка этапов тестирования для запуска, который определяет, какие шаги из пункта «Построение тестов» будут выполнены (если параметр не задан, запускаются все этапы, если задан — только указанные);
- ограничить доступные каждому контейнеру ресурсы: количество ядер процессора должно быть ограничено до $(1 + \text{НОМЕР_ВАРИАНТА} \% 2)$, что для 52 варианта составляет 1 ядро.

1. ОПИСАНИЕ DOCKERFILE

1.1. Описание Dockerfile для контейнера app

Написан Dockerfile_app (рисунок 1), который предназначен для сборки образа контейнера, в котором запускается веб-приложение на базе фреймворка Flask.

Базовым образом выбран ubuntu:22.04, согласно требованию задания. Все устанавливаемые пакеты фиксируются с конкретными версиями, что обеспечивает воспроизводимость сборки независимо от изменений в репозиториях дистрибутива. Определение актуальных версий пакетов в репозитории Ubuntu 22.04 выполнялось с помощью команды `apt-cache policy`

На первом этапе устанавливаются системные зависимости: система контроля версий Git (необходима для клонирования репозитория с приложением), интерпретатор Python и менеджер пакетов `pip`, утилита `patch` для применения изменений к исходному коду, а также OpenSSH-сервер для организации удаленного доступа. Установка всех пакетов выполнена одной командой `RUN` с объединением через `&&`, что позволяет разместить все слои `apt` в одном слое и уменьшить итоговый размер образа. После установки производится очистка кэша `apt`.

Рабочей директорией устанавливается `/app`. Затем выполняется клонирование репозитория `devops-examples` с ограничением глубины истории (`--depth 1`), что уменьшает объем скачиваемых данных. Из клонированного репозитория копируется содержимое папки `EXAMPLE_APP` в рабочую директорию, после чего временная папка с репозиторием удаляется.

Далее в образ копируются файлы публичного SSH-ключа и скрипта настройки SSH (рисунок 2). Скрипту даются права на выполнение, после чего он запускается. Скрипт `setup_ssh.sh` выполняет следующие действия: создает каталог `/run/sshd` (необходим для работы SSH-сервера), создает директорию `/root/.ssh` с правами 700, перемещает публичный ключ в `authorized_keys` и выставляет права 600, а также модифицирует конфигурационный файл SSH,

включая аутентификацию по ключам и отключая вход по паролю. После выполнения скрипт удаляется из образа.

В соответствии с требованием варианта задания, для контейнера app настроен SSH-доступ по публичному ключу. Аутентификация организована через ключи, размещенные в каталоге app/ssh/keys/. Публичный ключ копируется в образ при сборке и помещается в /root/.ssh/authorized_keys. SSH-сервер запускается в фоновом режиме параллельно с веб-приложением.

Затем копируются файл с Python-зависимостями requirements.txt, патч fix_host.patch и скрипт запуска entrypoint.sh. Выполняется установка зависимостей с помощью pip (с флагом --no-cache-dir для уменьшения размера образа), применение патча к файлу main.py и назначение прав на выполнение для скрипта запуска.

Команда CMD запускает скрипт entrypoint.sh, который в свою очередь запускает SSH-сервер в фоновом режиме и затем веб-приложение. Таким образом, основным процессом контейнера является Flask-приложение, а SSH-сервер работает как дополнительный сервис.

```
1 FROM ubuntu:22.04
2
3 RUN apt-get update && apt-get install -y --no-install-recommends \
4     git=1:2.34.1-1ubuntu1.17 \
5     python3=3.10.6-1-22.04.1 \
6     python3-pip=22.0.2+dfsg-1ubuntu0.7 \
7     patch=2.7.6-7build2 \
8     openssh-server=1:8.9p1-3ubuntu0.14 \
9     && apt-get clean \
10    && rm -rf /var/lib/apt/lists/*
11
12 WORKDIR /app
13
14 RUN git clone --depth 1 https://github.com/moevm/devops-examples.git /tmp/devops-examples && \
15     cp -r /tmp/devops-examples/EXAMPLE_APP/* /app/ && \
16     rm -rf /tmp/devops-examples
17
18 COPY app/ssh/keys/app_key.pub /tmp/
19 COPY app/ssh/setup_ssh.sh /tmp/
20
21 RUN chmod +x /tmp/setup_ssh.sh && \
22     bash /tmp/setup_ssh.sh && \
23     rm /tmp/setup_ssh.sh
24
25 COPY app/requirements.txt /app/
26 COPY patches/fix_host.patch /app/
27 COPY app/entrypoint.sh /app/
28
29 RUN pip3 install --no-cache-dir -r requirements.txt && \
30     patch main.py < fix_host.patch && \
31     chmod +x entrypoint.sh
32
33 CMD ["bash", "entrypoint.sh"]
```

Рисунок 1. Dockerfile_app

```

1  #!/bin/bash
2
3  mkdir -p /run/sshd
4  mkdir -p /root/.ssh && chmod 700 /root/.ssh
5
6  mv /tmp/app_key.pub /root/.ssh/authorized_keys
7  chmod 600 /root/.ssh/authorized_keys
8
9  sed -i 's/^#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config
10 sed -i 's/^#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config
11 | Ctrl+L to chat

```

Рисунок 2. Скрипт настройки SSH

1.2. Описание Dockerfile для контейнера tester

Написан Dockerfile_tester (рисунок 3), который предназначен для сборки образа контейнера, который выполняет автоматизированное тестирование веб-приложения.

Базовым образом также выбран ubuntu:22.04. Установка системных зависимостей включает Git (для клонирования репозитория с приложением), Python и pip, утилиту patch, а также Node.js и npm (необходимы для запуска инструментов проверки JavaScript-кода) и curl (для скачивания файлов). Все версии пакетов фиксированы. Установка выполнена одной командой RUN с последующей очисткой кэша. Определение актуальных версий пакетов в репозитории Ubuntu 22.04 выполнялось с помощью команды apt-cache policy

Рабочей директорией установлена /tester. Затем создается директория /app, в которую клонируется репозиторий с исходным кодом тестируемого приложения. Код копируется из временной папки в /app, после чего временные файлы удаляются. Наличие кода приложения в контейнере tester необходимо для выполнения статического анализа (pylint) и проверки форматирования JavaScript-файлов.

В контейнер копируются патч fix_host.patch (применяется к исходному коду приложения для корректной работы тестов), файл зависимостей requirements.txt, скрипт запуска entrypoint.sh и директория со скриптами тестов.

На следующем этапе устанавливаются Python-зависимости (pylint, requests), применяется патч к исходному коду приложения, назначаются права на выполнение скрипту запуска и всем скриптам в директории scripts. С помощью curl скачивается актуальная версия jslint.mjs с официального сайта

JSLint. Этот файл используется для проверки качества JavaScript-кода. Выбор прямой загрузки вместо использования устаревшего пакета pyjslint обусловлен неработоспособностью последнего (ошибка 404 при попытке скачать правила).

Команда CMD запускает скрипт entrypoint.sh, который с помощью exes запускает Python HTTP-сервер на порту 3000. Это делает HTTP-сервер корневым процессом контейнера, что соответствует требованию задания.

```
1 FROM ubuntu:22.04
2
3 RUN apt-get update && apt-get install -y --no-install-recommends \
4     git=1:2.34.1-lubuntu1.17 \
5     python3=3.10.6-1~22.04.1 \
6     python3-pip=22.0.2+dfsg-lubuntu0.7 \
7     patch=2.7.6-7build2 \
8     nodejs=12.22.9~dfsg-lubuntu3.6 \
9     npm=8.5.1~ds-1 \
10    curl=7.81.0-lubuntu1.23 \
11    && apt-get clean \
12    && rm -rf /var/lib/apt/lists/*
13
14 WORKDIR /tester
15
16 RUN mkdir -p /app && \
17     git clone --depth 1 https://github.com/moeym/devops-examples.git /tmp/devops-examples && \
18     cp -r /tmp/devops-examples/EXAMPLE_APP/* /app/ && \
19     rm -rf /tmp/devops-examples
20
21 COPY patches/fix_host.patch /tester/
22 COPY tester/requirements.txt /tester/
23 COPY tester/entrypoint.sh /tester/
24 COPY tester/scripts /tester/scripts
25
26 RUN pip3 install --no-cache-dir -r requirements.txt && \
27     curl -L https://www.jslint.com/jslint.mjs -o /tester/jslint.mjs && \
28     patch /app/main.py < /tester/fix_host.patch && \
29     chmod +x /tester/entrypoint.sh && \
30     chmod +x /tester/scripts/*
31
32 CMD ["bash", "entrypoint.sh"]
33
```

Рисунок 3. Dockerfile_tester

2. ОПИСАНИЕ СКРИПТОВ ЗАПУСКА ТЕСТОВ

2.1. Общий скрипт запуска тестов

Для последовательного выполнения всех этапов тестирования в контейнере tester реализован скрипт-оркестратор test_all.sh (рисунок 4).

Скрипт считывает переменную окружения TEST_STEPS, которая определяет набор тестов для запуска. Если переменная не задана или равна all, выполняются все доступные тесты. Если переменная содержит список этапов (например, pylint,integration), выполняются только указанные тесты.

В скрипте определены два массива: STEP_NAMES с названиями этапов тестирования (pylint, jslint, integration, pyjslint) и SCRIPT_NAMES с соответствующими именами скриптов.

Для каждого этапа выполняется проверка - должен ли он быть запущен на основе значения TEST_STEPS. Если этап пропускается, выводится сообщение [SKIP]. Если этап запускается, выводится сообщение [RUN], после чего выполняется соответствующий скрипт. По завершении скрипта проверяется код возврата: если он равен нулю, этап считается пройденным (выводится [PASS]), иначе - непройденным (выводится [FAIL]). В конце работы скрипт выводит статистику: количество пройденных тестов из общего числа запущенных.

```
1  #!/bin/bash
2
3  echo "Running tests with TEST_STEPS=${TEST_STEPS:-all}"
4
5  STEP_NAMES=("pylint" "jslint" "integration" "pyjslint")
6  SCRIPT_NAMES=("test_pylint.sh" "test_jslint.sh" "test_integration.sh" "test_pyjslint.sh")
7
8  PASSED=0
9  TOTAL=0
10
11 for i in "${!STEP_NAMES[@]}; do
12     STEP_NAME="${STEP_NAMES[$i]}"
13     SCRIPT_NAME="${SCRIPT_NAMES[$i]}"
14
15     if [[ "${TEST_STEPS:-all}" != "all" && "${TEST_STEPS}" != *"$STEP_NAME"* ]]; then
16         echo "[SKIP] $STEP_NAME (not in TEST_STEPS)"
17         continue
18     fi
19
20     TOTAL=$((TOTAL + 1))
21     echo "[RUN] $STEP_NAME"
22
23     bash "/tester/scripts/$SCRIPT_NAME"
24
25     if [ $? -eq 0 ]; then
26         echo "[PASS] $STEP_NAME"
27         PASSED=$((PASSED + 1))
28     else
29         echo "[FAIL] $STEP_NAME"
30     fi
31 done
32
33 echo "$PASSED / $TOTAL tests passed"
```

Рисунок 4. test_all.sh

2.2. Проверка бьютификакатором для JS

Был написан скрипт (рисунок 5), который создает директорию `/logs` и настраивает перенаправление потоков аналогично скрипту для `pylint`. Стандартный вывод направляется в `/logs/pyjslint_stdout.log`, поток ошибок — в `/logs/pyjslint_stderr.log`. Оба потока дублируются в консоль.

Скрипт находит все файлы с расширением `.js` в директории `/app`. Если файлы не найдены, выводится сообщение об ошибке и скрипт завершается с кодом 1.

Для демонстрации перенаправления потоков выводятся тестовые сообщения.

Согласно заданию, для проверки JavaScript кода предполагалось использовать инструмент `pyjslint` (JSLint wrapper). Однако при попытке запуска `pyjslint` возникает ошибка 404 при скачивании файла правил `jslint.js` с GitHub (рисунок 6). Причина ошибки заключается в том, что `pyjslint` при первом запуске пытается скачать файл правил `jslint.js` с GitHub-репозитория Дугласа Крокфорда. На момент выполнения работы данный URL недоступен (возвращает 404).

Поскольку использовать `pyjslint` в исходном виде невозможно, было принято решение реализовать собственную проверку с использованием оригинального `jslint` (Node.js-версия). Для этого был создан отдельный скрипт `test_jslint.sh` (рисунок 7), в котором команда `pyjslint` заменена на вызов `jslint` через Node.js с явным указанием файла правил.

```

1  #!/bin/bash
2
3  mkdir -p /logs
4
5  exec > >(tee -a /logs/pyjslint_stdout.log)
6  exec 2> >(tee -a /logs/pyjslint_stderr.log >&2)
7
8  echo "This is a pyjslint stderr message" >&2
9  echo "This is a pyjslint stdout message"
10
11  JS_FILES=$(find /app -name "*.js")
12
13  if [ -z "$JS_FILES" ]; then
14      echo "ERROR: No JS files found in /app" >&2
15      exit 1
16  fi
17
18  pyjslint $JS_FILES
19
20  exit $?

```

Рисунок 5. test_pyjslint.sh

```

~/Documents/Development/devops-1h2026/1306 ShavrinAP/src | 1306 ShavrinAP !11 73 .....
docker compose exec tester bash /tester/scripts/test_pyjslint.sh
This is a pyjslint stdout message
This is a pyjslint stderr message
Traceback (most recent call last):
  File "/usr/local/bin/pyjslint", line 8, in <module>
    sys.exit(main())
  File "/usr/local/lib/python3.10/dist-packages/pyjslint.py", line 123, in main
    output, valid = process(open(filename, "r"), options)
  File "/usr/local/lib/python3.10/dist-packages/pyjslint.py", line 95, in process
    lint = get_lint(options)
  File "/usr/local/lib/python3.10/dist-packages/pyjslint.py", line 77, in get_lint
    response = urlopen('https://raw.githubusercontent.com/douglascrockford/'
  File "/usr/lib/python3.10/urllib/request.py", line 216, in urlopen
    return opener.open(url, data, timeout)
  File "/usr/lib/python3.10/urllib/request.py", line 525, in open
    response = meth(req, response)
  File "/usr/lib/python3.10/urllib/request.py", line 634, in http_response
    response = self.parent.error(
  File "/usr/lib/python3.10/urllib/request.py", line 557, in error
    result = self._call_chain(*args)
  File "/usr/lib/python3.10/urllib/request.py", line 496, in _call_chain
    result = func(*args)
  File "/usr/lib/python3.10/urllib/request.py", line 749, in http_error_302
    return self.parent.open(new, timeout=req.timeout)
  File "/usr/lib/python3.10/urllib/request.py", line 525, in open
    response = meth(req, response)
  File "/usr/lib/python3.10/urllib/request.py", line 634, in http_response
    response = self.parent.error(
  File "/usr/lib/python3.10/urllib/request.py", line 563, in error
    return self._call_chain(*args)
  File "/usr/lib/python3.10/urllib/request.py", line 496, in _call_chain
    result = func(*args)
  File "/usr/lib/python3.10/urllib/request.py", line 643, in http_error_default
    raise HTTPError(req.full_url, code, msg, hdrs, fp)
urllib.error.HTTPError: HTTP Error 404: Not Found

```

Рисунок 6. Ошибка test_pyjslint.sh


```

1  #!/bin/bash
2
3  mkdir -p /logs
4
5  exec > >(tee -a /logs/jslint_stdout.log)
6  exec 2> >(tee -a /logs/jslint_stderr.log >&2)
7
8  echo "This is a jslint stderr message" >&2
9  echo "This is a jslint stdout message"
10
11 JS_FILES=$(find /app -name "*.js")
12
13 if [ -z "$JS_FILES" ]; then
14     echo "ERROR: No JS files found in /app" >&2
15     exit 1
16 fi
17
18 node ./jslint.mjs $JS_FILES
19
20 exit $?

```

Рисунок 7. test_jslint.sh

2.3. Статический анализ по 10 существующим критери

Был написан скрипт (рисунок 8), который создает директорию /logs для хранения логов, после чего с помощью команд exes перенаправляет стандартный вывод и поток ошибок. Стандартный вывод направляется в файл /logs/pylint_stdout.log и одновременно в консоль (через tee -a). Поток ошибок направляется в файл /logs/pylint_stderr.log и также дублируется в консоль, но уже в поток ошибок (>&2).

Затем скрипт находит все файлы с расширением .py в директории /app. Если файлы не найдены, выводится сообщение об ошибке и скрипт завершается с кодом 1.

Для демонстрации перенаправления потоков выводятся тестовые сообщения.

Запускается pylint с отключением всех проверок (--disable=all) и включением только десяти выбранных критериев (--enable=...) (таблица 1)

Все проверяемые файлы передаются в pylint через переменную \$TARGET_FILES. Код возврата pylint передается как код возврата скрипта.

Таблица 1. Критерии проверки

Код	Название	Описание
C0114	missing-module-docstring	Отсутствие документации модуля
C0116	missing-function-docstring	Отсутствие документации функции
C0304	missing-final-newline	Отсутствие пустой строки в конце файла
C0413	wrong-import-position	Импорт расположен не в начале файла
C3001	unnecessary-lambda-assignment	Бесполезное присваивание лямбды переменной
W0601	global-variable-undefined	Использование <code>global</code> для неопределенной переменной
W0612	unused-variable	Неиспользуемая переменная
W0613	unused-argument	Неиспользуемый аргумент функции
W0622	redefined-builtin	Переопределение встроенной функции
E0602	undefined-variable	Использование неопределенной переменной

```

1  #!/bin/bash
2
3  mkdir -p /logs
4  exec 1> >(tee -a /logs/pylint_stdout.log)
5  exec 2> >(tee -a /logs/pylint_stderr.log >&2)
6
7  echo "This is a pylinttest stderr message" >&2
8  echo "This is a pylinttest stdout message"
9
10 TARGET_FILES=$(find /app -name "*.py")
11
12 if [ -z "$TARGET_FILES" ]; then
13     echo "ERROR: No Python files found in /app" >&2
14     exit 1
15 fi
16
17 pylint --disable=all \
18     --enable=C0114,C0116,C0304,C0413,C3001,W0601,W0612,W0613,W0622,E0602 \
19     $TARGET_FILES
20
21 exit $?

```

Рисунок 8. test_pylint.sh

2.4. Интеграционный тест кодов возврата

Был написан скрипт `test_integration.sh` (рисунок 9), который создает директорию `/logs` и настраивает перенаправление потоков. Затем он запускает Python-скрипт `test_integration.py` (рисунок 10).

GET-запросы:

- `/` — главная страница (ожидается 200)
- `/files` — страница со списком файлов (200)
- `/upload` — страница загрузки (200)
- `/notexist` — несуществующий маршрут (404)
- `/to_files` — редирект на `/files` (200 после редиректа)
- `/success/test` — страница успеха (200)

POST-запросы к `/login`:

- с правильными учетными данными (`admin/password`) — ожидается 302 (редирект)
- с неправильными учетными данными — ожидается 401 (Unauthorized)

Для каждого запроса функция `test_path` отправляет HTTP-запрос с помощью библиотеки `requests`. Для POST-запросов передаются данные формы. Параметр `allow_redirects=False` используется для проверки кода редиректа без автоматического перехода.

При успешной проверке выводится сообщение OK: метод путь -> код. При неуспешной проверке выводится сообщение FAIL и ошибка записывается в поток ошибок. В случае исключения (например, недоступность сервера) выводится сообщение ERROR.

```
1  #!/bin/bash
2
3  mkdir -p /logs
4  exec > >(tee -a /logs/integration_stdout.log)
5  exec 2> >(tee -a /logs/integration_stderr.log >&2)
6
7  echo "This is a integration stderr message" >&2
8  echo "This is a pylinintegrationttest stdout message"
9
10 python3 /tester/scripts/test_integration.py
11 exit $?

```

Рисунок 9. `test_integration.sh`

```

1 import requests
2 import sys
3
4 URL_BASE = "http://app:5000"
5
6 def test_path(path, expected_status, method="GET", data=None, allow_redirects=False):
7     try:
8         if method == "GET":
9             resp = requests.get(URL_BASE + path, allow_redirects=allow_redirects)
10        elif method == "POST":
11            resp = requests.post(URL_BASE + path, data=data, allow_redirects=allow_redirects)
12        else:
13            return False
14
15        status = resp.status_code
16        if status == expected_status:
17            print(f"OK: {method} {path} -> {status}")
18            return True
19        else:
20            print(f"FAIL: {method} {path} -> {status} (expected {expected_status})", file=sys.stderr)
21            return False
22    except Exception as e:
23        print(f"ERROR: {method} {path} -> {e}", file=sys.stderr)
24        return False
25
26 def main():
27     failed = 0
28
29     # GET
30     tests = [
31         ("/", 200),
32         ("/files", 200),
33         ("/upload", 200),
34         ("/notexist", 404),
35         ("/to_files", 200),
36         ("/success/test", 200),
37     ]
38
39     for path, expected in tests:
40         if not test_path(path, expected):
41             failed += 1
42
43     # POST /login (успешный)
44     if test_path("/login", 302, "POST", {"name": "admin", "password": "password"}):
45         pass
46     else:
47         failed += 1
48
49     # POST /login (неуспешный)
50     if test_path("/login", 401, "POST", {"name": "wrong", "password": "wrong"}):
51         pass
52     else:
53         failed += 1
54
55     if failed == 0:
56         print("All integration tests passed")
57         sys.exit(0)
58     else:
59         print(f"{failed} integration test(s) failed")
60         sys.exit(1)
61
62 if __name__ == "__main__":
63     main()

```

Рисунок 10. test_integration.py

3. ОПИСАНИЕ КОНФИГУРАЦИИ DOCKER-COMPOSE

Была определена конфигурация Docker Compose, включающая два сервиса: `app` и `tester` (рисунок 11).

Сервис `app` предназначен для запуска веб-приложения на основе Flask, полученного из репозитория `devops-examples`. Сборка образа осуществляется по `Dockerfile`, расположенному по пути `app/Dockerfile_app`. Контекст сборки задается относительно корневой директории проекта (`.`), что позволяет включать в процесс сборки общие ресурсы, такие как каталог `patches`. Для обеспечения доступа к приложению с хост-машины настроен проброс портов:

- `127.0.0.1:8080:5000` — доступ к веб-интерфейсу Flask (внутренний порт 5000, внешний 8080 на `localhost`);
- `127.0.0.1:2222:22` — доступ по SSH (внутренний порт 22, внешний 2222).

Сборка образа `tester` осуществляется по `Dockerfile tester/Dockerfile_tester`. Контекст сборки также задается корневой директорией проекта, что позволяет копировать общие патчи и скрипты.

Для взаимодействия с сервисом `app` используется внутренняя сеть Docker Compose - обращение к приложению происходит по имени сервиса `app:5000`. Это обеспечивает изоляцию и не требует публикации портов на хост-машине. Тем не менее, порт 3000 проброшен на `127.0.0.1:3000` для отладки HTTP-сервера, который является корневым процессом контейнера (в соответствии с требованием задания).

В конфигурацию сервиса `tester` добавлена директория `volumes`, монтирующая локальную папку `./tester/logs` в `/logs` внутри контейнера. Это позволяет сохранять логи тестирования на хост-машине для последующего анализа.

Также сервис `tester` имеет зависимость от сервиса `app`, заданную директивой `depends_on`. Это гарантирует, что контейнер с веб-приложением будет запущен до начала выполнения тестов.

Для управления поведением тестов используется файл `.env`, расположенный в корневой директории `src`. В нем задается переменная `TEST_STEPS`, определяющая набор запускаемых тестов.

Допустимые значения:

- `all` (или пустое значение) — запуск всех тестов;
- `pylint` — только статический анализ Python;
- `pyjslint` или `jslint` — только проверка JavaScript;
- `integration` — только интеграционные тесты;
- комбинации через запятую, например `pylint,integration`.

Переменная передается в контейнер через параметр `environment`.

В соответствии с вариантом задания, для обоих сервисов задано ограничение по количеству ядер процессора: `cpu_count: 1`.

```
1  services:
2    app:
3      build:
4        context: .
5        dockerfile: app/Dockerfile_app
6      ports:
7        - "127.0.0.1:8080:5000"
8        - "127.0.0.1:2222:22"
9      cpu_count: 1
10     restart: unless-stopped
11
12     tester:
13       build:
14         context: .
15         dockerfile: tester/Dockerfile_tester
16       ports:
17         - "127.0.0.1:3000:3000"
18       depends_on:
19         - app
20       volumes:
21         - ./tester/logs:/logs
22       environment:
23         - TEST_STEPS=${TEST_STEPS:-all}
24       cpus: 1
25       restart: unless-stopped
```

Рисунок 11. `docker-compose.yaml`

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была разработана и реализована docker-compose конфигурация для автоматизации тестирования веб-приложения в соответствии с вариантом №52. Разработаны Dockerfile для двух контейнеров: app с веб-приложением на Flask, включающий настройку SSH-доступа по публичному ключу и применение патча для корректировки хоста, и tester с набором инструментов для тестирования. Реализованы три типа тестов: статический анализ Python-кода по десяти критериям с использованием pylint, проверка форматирования JavaScript-кода с использованием jslint и интеграционные тесты, проверяющие коды возврата HTTP-запросов. Настроено логирование тестов в соответствии с требованием задания: каждый этап тестирования выводится в стандартные потоки вывода и ошибок, а также сохраняется в отдельные файлы для stdout и stderr. Организован SSH-доступ в контейнер app по публичному ключу с пробросом порта на хост-машину. Реализовано управление набором запускаемых тестов через переменную окружения TEST_STEPS, задаваемую в файле .env, что позволяет гибко выбирать необходимые этапы тестирования. Выполнены требования по ограничению ресурсов: для обоих контейнеров установлено ограничение в одно ядро процессора. Соблюдены общие требования к оформлению Dockerfile и docker-compose, включая фиксацию версий пакетов, объединение команд установки в один RUN, проброс портов только на localhost с номерами выше 1024 и отказ от использования сети HOST.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Docker Documentation: Dockerfile reference [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/reference/dockerfile/> (дата обращения: 31.03.2026).
2. Docker Documentation: Docker Comp overview [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/compose/> (дата обращения: 31.03.2026).
3. PyPI: pylint [Электронный ресурс]. – Режим доступа: <https://pypi.org/project/pylint/> (дата обращения: 01.04.2026).
4. PyPI: requests [Электронный ресурс]. – Режим доступа: <https://pypi.org/project/requests/> (дата обращения: 01.04.2026).
5. PyPI: pyjslint [Электронный ресурс]. – Режим доступа: <https://pypi.org/project/pyjslint/> (дата обращения: 02.04.2026).
6. GitHub: jslint [Электронный ресурс]. – Режим доступа: <https://github.com/bramstein/jslint> (дата обращения: 02.04.2026).
7. Пример веб-приложения для курса DevOps [Электронный ресурс]. – Режим доступа: <https://github.com/moevm/devops-examples> (дата обращения: 30.03.2026).